



Computer Organization and Assembly Language (EL-2003)

Semester: Fall 2025

Section: Section: BSSE-3A - BSSE-3B

Course Instructor: Muhammad Nadeem Ghouri

LAB # 08

Stack Operations & Procedures

Lab Objectives:

By the end of this lab, students will be able to:

- Understand the concept and working of the Runtime Stack in assembly language and its role in function calls.
- Learn the purpose and functionality of the PUSH instruction for storing data temporarily on the stack.
- Explore the usage of the POP instruction to retrieve data from the stack efficiently.
- Understand the PROC directive and its significance in defining modular procedures in assembly language.
- Implement CALL and RET instructions to control program flow and manage procedure calls and returns.
- Develop and execute programs using nested procedures, demonstrating proper stack management and parameter passing.
- Enhance problem-solving and logical thinking skills through structured and modular assembly programming.

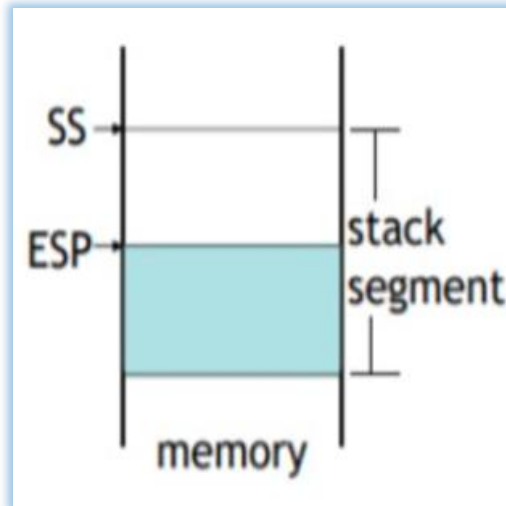
STACK

- LIFO (Last-In, First-Out) data structure.
- PUSH/ POP operations
- You probably have had experiences on implementing it in high-level languages.
- Here, we concentrate on runtime stack, directly supported by hardware in the
- CPU. It is essential for calling and returning from procedures.

RUNTIME STACK

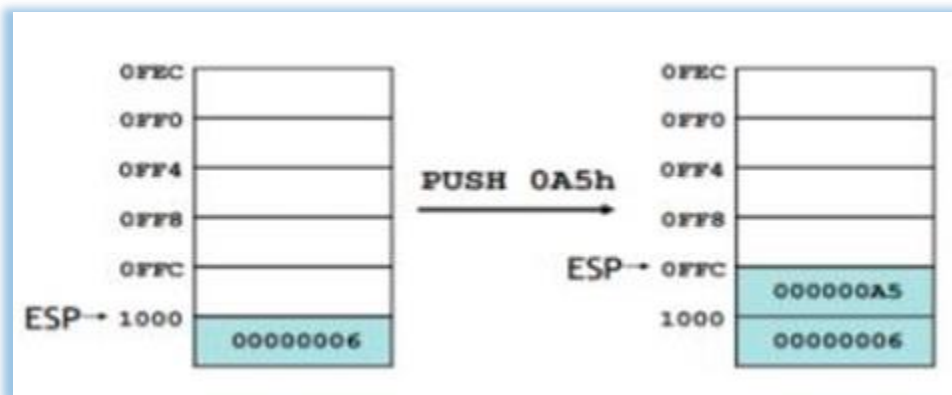
Managed by the CPU, using two registers

- SS (stack segment)
- ESP (stack pointer): point the last value to be added to, or pushed on, the top of stack usually modified by instructions:
 - **CALL, RET, PUSH and POP**

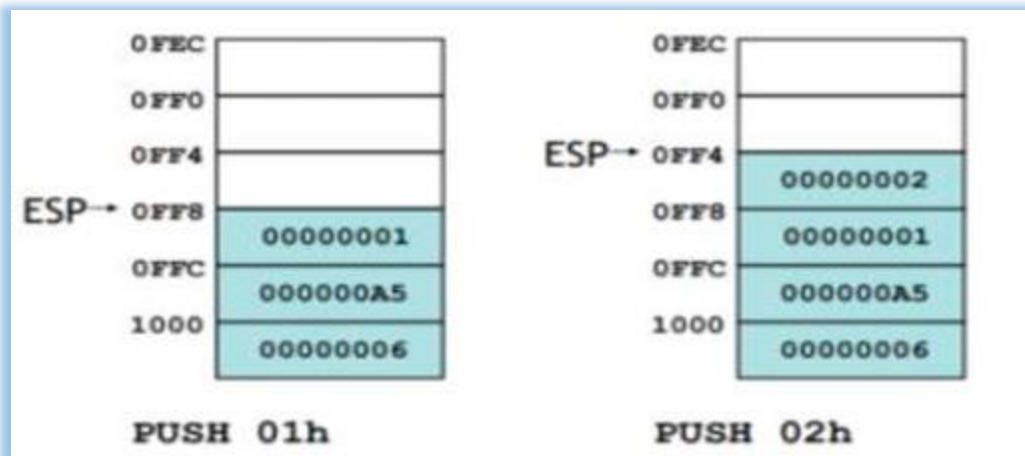


PUSH OPERATION

Push Operation A 32-bit push operation decrements the stack pointer by 4 and copies a value into the location in the stack pointed to by the stack pointer.

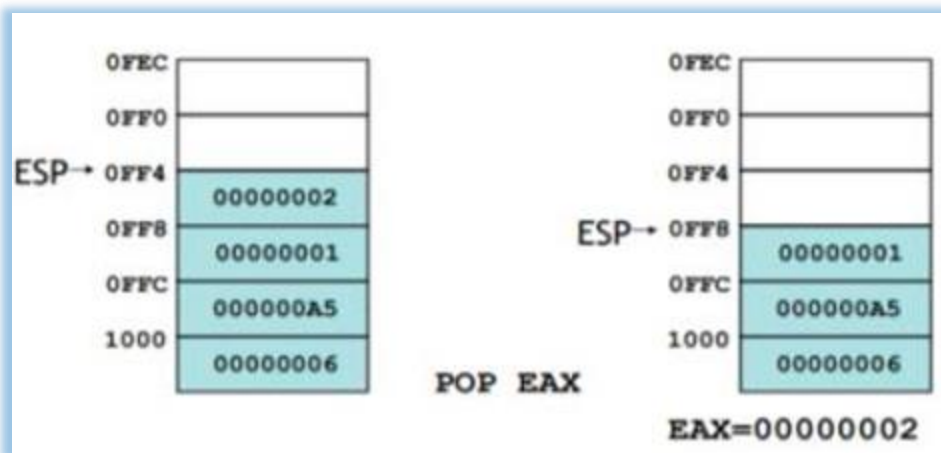


The same stack after pushing two more integers:



POP OPERATION

Pop Operation A pop operation removes a value from the stack. After the value is popped from the stack, the stack pointer is incremented (by the stack element size) to point to the next- highest location in the stack. It copies value at stack [ESP] into a register or variable



PUSH and POP INSTRUCTIONS

PUSH syntax:

- PUSH r/m16
- PUSH r/m32
- PUSH imm32

POP syntax:

- POP r/m16
- POP r/m32

EXAMPLE # 01:

STACK AND NESTED LOOPS

```

1  INCLUDE Irvine32.inc
2  .data
3  msg1 BYTE "Example: Stack and Nested Loops", 0
4  msg2 BYTE "Final value of EBX after nested loops: ", 0
5
6  .code
7  main PROC
8      mov ebx, 0
9      mov ecx, 5
10
11  L1:
12      push ecx
13      mov ecx, 10
14
15  L2:
16      inc ebx
17      loop L2
18
19      pop ecx
20      loop L1
21
22      call CrLf
23      mov edx, OFFSET msg1
24      call WriteString
25      call CrLf
26      mov edx, OFFSET msg2
27      call WriteString
28      mov eax, ebx
29      call WriteDec
30      call CrLf
31
32  exit
33  main ENDP
34  END main

```

Microsoft Visual Studio Debug Console

```

Example: Stack and Nested Loops
Final value of EBX after nested loops: 50

C:\Users\Muhammad Nadeem\source\repos\Sunday\Debug\Sunday.exe (process 7916) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

```

EXAMPLE # 02:**HOW VALUES ARE TEMPORARILY STORED AND RETRIEVED FROM THE STACK DURING ARITHMETIC OPERATIONS.**

You are a **Software Engineering student** developing a calculator program that needs to temporarily store intermediate results during arithmetic computations.

Write an Assembly program that:

1. Pushes two numbers onto the stack.
2. Pops them back one by one.
3. Adds them and displays the sum.
4. Again pushes the result, then pops and displays it as the **final stored result**.

```

1  INCLUDE Irvine32.inc
2  .data
3  num1 WORD 15
4  num2 WORD 25
5  msg1 BYTE "Initial Numbers: ", 0
6  msg2 BYTE "Sum after POP (A + B): ", 0
7  msg3 BYTE "Final Result after PUSH & POP: ", 0
8  space BYTE " ", 0
9
10 .code
11 main PROC
12     call CrLf
13     mov edx, OFFSET msg1
14     call WriteString
15     call CrLf
16
17     movzx eax, num1
18     call WriteDec
19     mov edx, OFFSET space
20     call WriteString
21     movzx eax, num2
22     call WriteDec
23     call CrLf
24
25     mov ax, num1
26     push ax
27     mov ax, num2
28     push ax
29
30     pop bx
31     pop ax
32     add ax, bx

```

```

33
34     call CrLf
35     mov edx, OFFSET msg2
36     call WriteString
37     movzx eax, ax
38     call WriteDec
39     call CrLf
40
41     push ax
42     pop bx
43
44     call CrLf
45     mov edx, OFFSET msg3
46     call WriteString
47     movzx eax, bx
48     call WriteDec
49     call CrLf
50
51     exit
52 main ENDP
53 END main

```

```

Initial Numbers:
15 25

Sum after POP (A + B): 40

Final Result after PUSH & POP: 40

C:\Users\Muhammad Nadeem\source\repos\Sunday\Debug\S

```

EXAMPLE # 03:

DEMONSTRATES HOW STACK-BASED MEMORY MANAGEMENT SUPPORTS REAL-WORLD FEATURES LIKE UNDO/REDO OR VERSION ROLLBACK. REINFORCES UNDERSTANDING OF LAST-IN, FIRST-OUT (LIFO) PRINCIPLE CRUCIAL FOR RECURSION, FUNCTION CALLS, AND BACKTRACKING ALGORITHMS.

- Store 5 version numbers in an array (representing software updates).
- Push all version numbers onto the stack (latest version last).
- Pop them into another array in reverse order, simulating a rollback feature.
- Display both arrays to show before and after rollback versions.

```

1  INCLUDE Irvine32.inc
2  .data
3  versionHistory WORD 101, 102, 103, 104, 105
4  rollbackHistory WORD 5 DUP(?)
5  msg1 BYTE "Version History (Latest Last):", 0
6  msg2 BYTE "Rollback Order (After Using PUSH & POP):", 0
7  space BYTE " ", 0
8
9  .code
10 main PROC
11     call    Crlf
12     mov     edx, OFFSET msg1
13     call    WriteString
14     call    Crlf
15

```

Pop all version numbers in reverse order

```

37     mov     ecx, LENGTHOF rollbackHistory
38     mov     edi, OFFSET rollbackHistory
39     pop_versions:
40         pop     ax
41         mov     [edi], ax
42         add     edi, TYPE rollbackHistory
43         loop    pop_versions
44
45     call    Crlf
46     mov     edx, OFFSET msg2
47     call    WriteString
48     call    Crlf

```

Display original version history

```

17     mov     ecx, LENGTHOF versionHistory
18     mov     esi, OFFSET versionHistory
19     display_original:
20         movzx  eax, WORD PTR [esi]
21         call    WriteDec
22         mov     edx, OFFSET space
23         call    WriteString
24         add     esi, TYPE versionHistory
25         loop    display_original

```

Display rollback array (reversed)

```

51     mov     ecx, LENGTHOF rollbackHistory
52     mov     esi, OFFSET rollbackHistory
53     display_rollback:
54         movzx  eax, WORD PTR [esi]
55         call    WriteDec
56         mov     edx, OFFSET space
57         call    WriteString
58         add     esi, TYPE rollbackHistory
59         loop    display_rollback
60
61     call    Crlf
62     exit
63 main ENDP
64 END main

```

Push all version numbers to stack

```

28     mov     ecx, LENGTHOF versionHistory
29     mov     esi, OFFSET versionHistory
30     push_versions:
31         mov     ax, [esi]
32         push    ax
33         add     esi, TYPE versionHistory
34         loop    push_versions
35

```

```

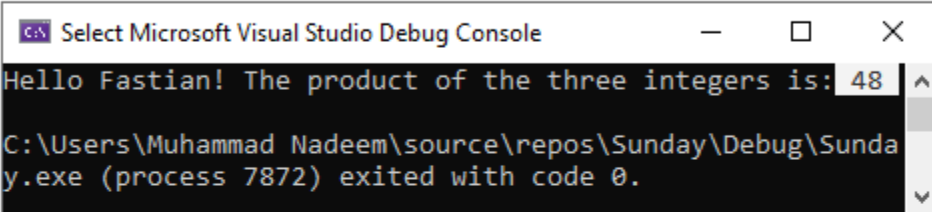
Version History (Latest Last):
101 102 103 104 105
Rollback Order (After Using PUSH & POP):
105 104 103 102 101
C:\Users\Muhammad Nadeem\source\repos\Sunday

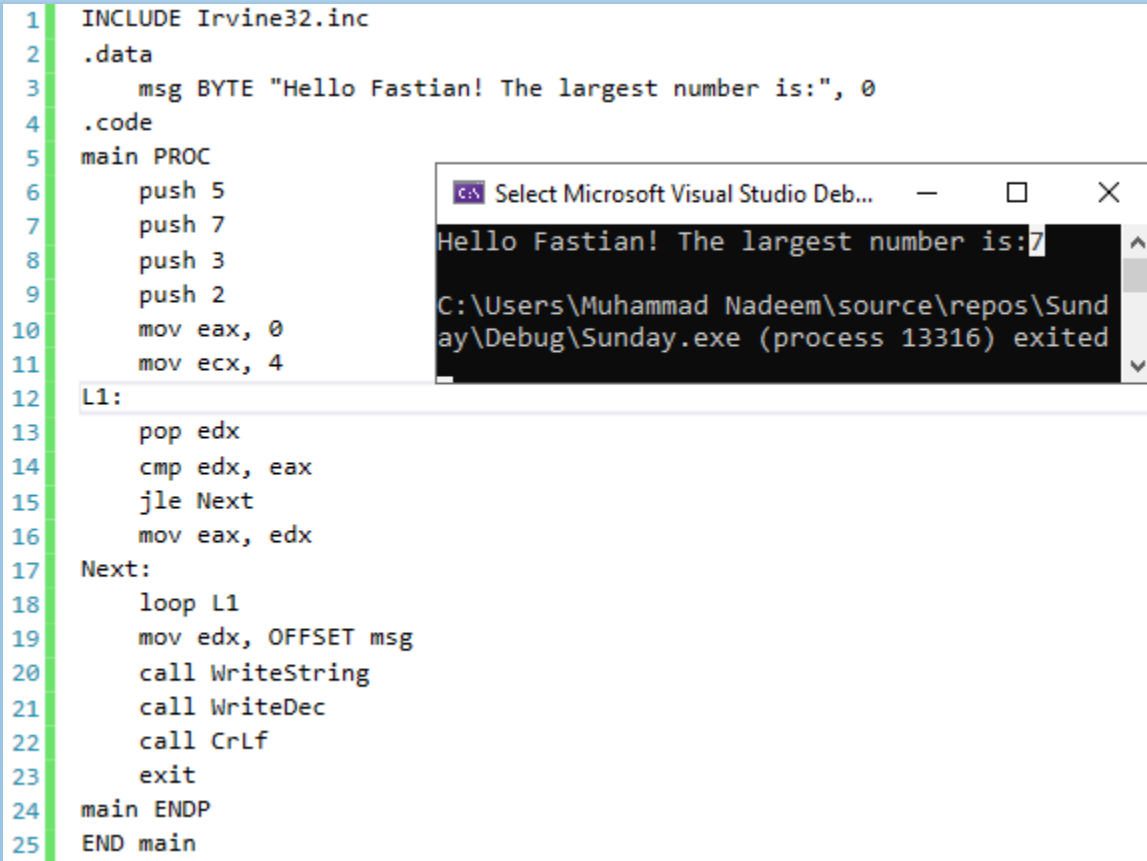
```

EXAMPLE # 04:

THIS PROGRAM PUSHES THREE INTEGERS ONTO THE STACK AND THEN POPS THEM TO COMPUTE THEIR PRODUCT.

```
1  INCLUDE Irvine32.inc
2  .data
3      multp DWORD 2
4      msg BYTE "Hello Fastian! The product of the three integers is: ", 0
5  .code
6  main PROC
7      mov eax, 1
8      mov ecx, 3
9  PushLoop:
10     push multp
11     add multp, 2
12     loop PushLoop
13     mov ecx, 3
14  MultiplyLoop:
15     pop ebx
16     mul ebx
17     loop MultiplyLoop
18     mov edx, OFFSET msg
19     call WriteString
20     call WriteDec
21     call CrLf
22     exit
23  main ENDP
24  END main
```



EXAMPLE # 05:**TO FIND THE LARGEST NUMBER THROUGH A STACK.**


```

1  INCLUDE Irvine32.inc
2  .data
3      msg BYTE "Hello Fastian! The largest number is:", 0
4  .code
5  main PROC
6      push 5
7      push 7
8      push 3
9      push 2
10     mov eax, 0
11     mov ecx, 4
12 L1:
13     pop edx
14     cmp edx, eax
15     jle Next
16     mov eax, edx
17 Next:
18     loop L1
19     mov edx, OFFSET msg
20     call WriteString
21     call WriteDec
22     call CrLf
23     exit
24 main ENDP
25 END main

```

Output in console window:
Hello Fastian! The largest number is: 7
C:\Users\Muhammad Nadeem\source\repos\Sunday\Debug\Sunday.exe (process 13316) exited

PUSHFD and POPFD INSTRUCTIONS

The MOV instruction cannot be used to copy the flags to a variable.

The PUSHFD instruction pushes the 32-bit EFLAGS register on the stack, and POPFD pops the stack into EFLAGS:

PUSHFD

POPFD

EXAMPLE # 06:**TO FIND THE LARGEST NUMBER THROUGH A STACK.**

```

1  INCLUDE Irvine32.inc
2  .data
3      msg1 BYTE "Original Flags saved on stack.", 0
4      msg2 BYTE "Flags restored from stack.", 0
5  .code
6  main PROC
7      mov eax, 5
8      sub eax, 5
9      pushfd
10     mov edx, OFFSET msg1
11     call WriteString
12     call CrLf
13     mov eax, 10
14     add eax, 1
15     popfd
16     mov edx, OFFSET msg2
17     call WriteString
18     call CrLf
19     call DumpRegs
20     exit
21 main ENDP
22 END main

```

Microsoft Visual Studio Debug Console

```

Original Flags saved on stack.
Flags restored from stack.

EAX=0000000B  EBX=00C1F000  ECX=001A10AA  EDX=001A601F
ESI=001A10AA  EDI=001A10AA  EBP=00EFFF34  ESP=00EFFF28
EIP=001A3695  EFL=00000202  CF=0  SF=0  ZF=0  OF=0  AF=0  PF=0

```

Explanation:

1. After `sub eax, 5`, Zero Flag (ZF) becomes 1.
2. `PUSHFD` saves this flag state to the stack.
3. Some operations are performed that modify flags (e.g., `add eax, 1`).
4. `POPFD` restores the original flags from the stack — so `ZF=1` again, exactly as before the modification.

`PUSHFD` and `POPFD` are used to preserve and restore the CPU's status flags, ensuring that temporary operations don't unintentionally affect conditional logic or program flow.

PROCEDURES

- Procedures or subroutines are very important in assembly language, as the assembly language programs tend to be large in size.
- Procedures are identified by a name. Following this name, the body of the procedure is described which performs a well-defined job.
- End of the procedure is indicated by a return statement.

PROC Directive

We can define a procedure as a named block of statements that ends in a return statement. A procedure is declared using the **PROC** and **ENDP** directives. It must be assigned a name (a valid identifier). When we create a procedure other than your program's startup procedure, end it with a **RET** instruction. **RET** forces the CPU to return to the location from where the procedure was called:

Let us say, sample is the name of procedure.

```
sample PROC
.
.
ret
sample ENDP
```

The procedure is called from another function by using the **CALL** instruction. The **CALL** instruction should have the name of the called procedure as an argument as shown below

CALL Sample

The called procedure returns the control to the calling procedure by using the **RET** instruction.

Call & RET INSTRUCTIONS

CALL instruction is used whenever we need to make a call to some procedure or a subprogram. Whenever a **CALL** is made, the following process takes place inside the microprocessor:

- The address of the next instruction that exists in the caller program (after the program **CALL** instruction) is stored in the stack.
- The instruction queue is emptied for accommodating the instructions of the procedure. Then, the contents of the instruction pointer (IP) is changed with the address of the first instruction of the procedure.
- The subsequent instructions of the procedure are stored in the instruction queue for execution.
- The Syntax for the **CALL** instruction is mentioned above.

RET instruction stands for return. This instruction is used at the end of the procedures or the subprograms. This instruction transfers the execution to the caller program. Whenever the RET instruction is called, the following process takes place inside the microprocessor:

- The address of the next instruction in the mainline program which was previously stored inside the stack is now again fetched and is placed inside the instruction pointer (IP).
- The instruction queue will now again be filled with the subsequent instructions of the mainline program.

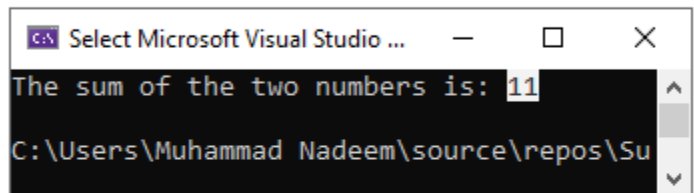
EXAMPLE # 07:

ADD TWO NUMBERS USING PROCEDURES

```

1  INCLUDE Irvine32.inc
2  .data
3      var1 DWORD 5
4      var2 DWORD 6
5      msg BYTE "The sum of the two numbers is: ", 0
6  .code
7  main PROC
8      call AddTwo
9      mov edx, OFFSET msg
10     call WriteString
11     call WriteDec
12     call CrLf
13     exit
14 main ENDP
15
16 AddTwo PROC
17     mov eax, var1
18     add eax, var2
19     ret
20 AddTwo ENDP
21 END main

```



EXAMPLE # 08:**THE SUM OF INTEGERS USING PROCEDURES**

```

1  INCLUDE Irvine32.inc
2
3  INTEGER_COUNT = 3
4
5  .data
6      str1 BYTE "Enter a signed integer: ", 0
7      str2 BYTE "The sum of the integers is: ", 0
8      array DWORD INTEGER_COUNT DUP(?)
9
10 .code
11 main PROC
12     call Clrscr
13     mov esi, OFFSET array
14     mov ecx, INTEGER_COUNT
15     call PromptForIntegers
16     call ArraySum
17     call DisplaySum
18     exit
19 main ENDP
20

```

```

21 PromptForIntegers PROC USES ecx edx esi
22     mov edx, OFFSET str1
23 L1:
24     call WriteString
25     call ReadInt
26     call Crlf
27     mov [esi], eax
28     add esi, TYPE DWORD
29     loop L1
30     ret
31 PromptForIntegers ENDP
32
33 ArraySum PROC USES esi ecx
34     mov eax, 0
35 L1:
36     add eax, [esi]
37     add esi, TYPE DWORD
38     loop L1
39     ret
40 ArraySum ENDP

```

```

42 DisplaySum PROC USES edx
43     mov edx, OFFSET str2
44     call WriteString
45     call WriteInt
46     call Crlf
47     ret
48 DisplaySum ENDP
49
50 END main

```

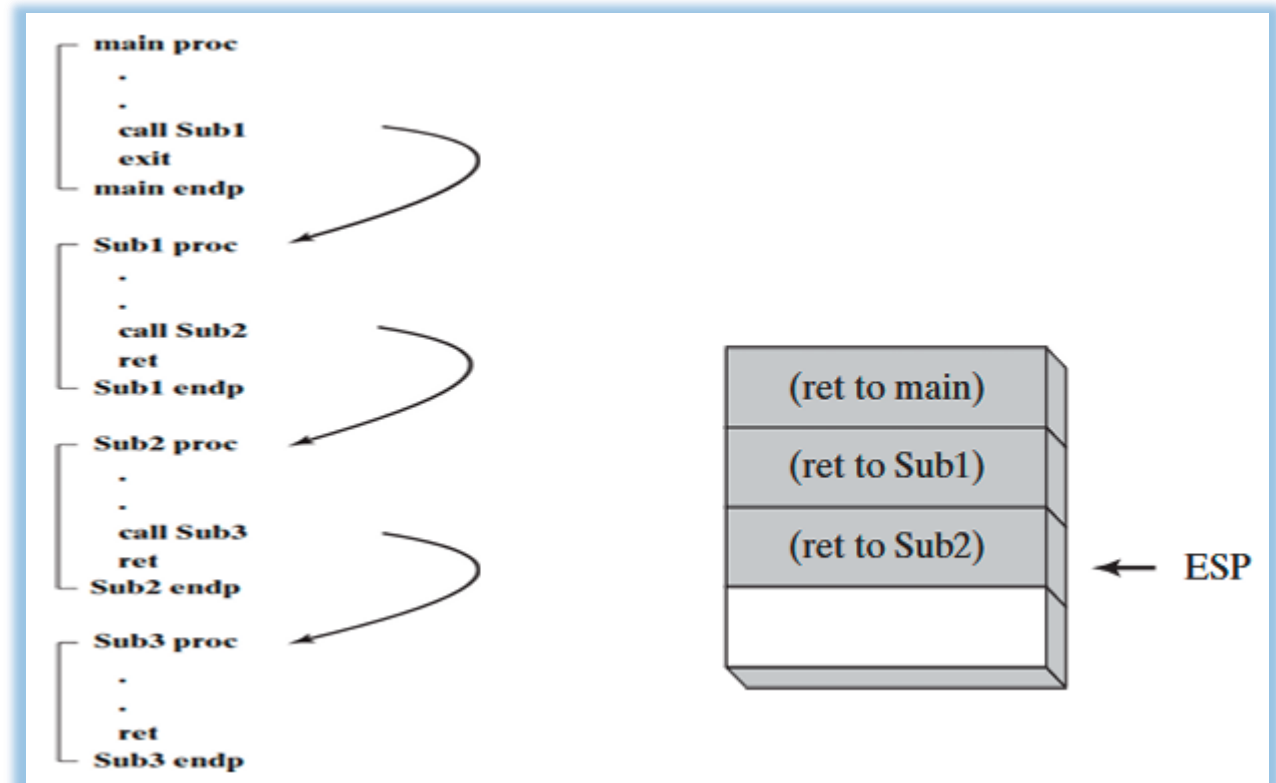
```

Enter a signed integer: 5
Enter a signed integer: 10
Enter a signed integer: 15
The sum of the integers is: +30
C:\Users\Muhammad Nadeem\source\repos\Sunday\De
(gcc.exe 8.2.2) -o sum.exe -

```

NESTED PROCEDURE CALLS

A nested procedure call occurs when a called procedure calls another procedure before the first procedure returns.



EXAMPLE # 09:**HOW NESTED PROCEDURE CALLS WORK**

A **nested procedure call** occurs when a **procedure calls another procedure** before returning to the procedure that called it.

Think of it like **one function calling another** before finishing its own work.

```

1  INCLUDE Irvine32.inc
2  .data
3      var1 DWORD 5
4      var2 DWORD 6
5      msg1 BYTE "The sum calculated in AddTwo is: ", 0
6      msg2 BYTE "Values printed inside AddTwo1:", 0
7  .code
8  main PROC
9      call AddTwo
10     call CrLf
11     exit
12 main ENDP
13
14 AddTwo PROC
15     mov eax, var1
16     mov ebx, var2
17     add eax, var2
18
19     mov edx, OFFSET msg1
20     call WriteString
21     call WriteInt
22     call CrLf
23
24     call AddTwo1
25     ret
26 AddTwo ENDP

```

```

28 AddTwo1 PROC
29     mov ecx, var1
30     mov edx, var2
31
32     mov ebx, OFFSET msg2
33     call WriteString
34     call CrLf
35
36     mov eax, ecx
37     call WriteInt
38     call CrLf
39
40     mov eax, edx
41     call WriteInt
42     call CrLf
43     ret
44 AddTwo1 ENDP
45 END main

```

```

The sum calculated in AddTwo is: +11
C:\Users\Muhammad Nadeem\source\repos\Sunday
\Debug\Sunday.exe (process 15276) exited wit

```

(How It Works):

- 1 main starts and calls AddTwo → return address is pushed on stack.
- 2 AddTwo adds var1 and var2, prints the sum, then calls AddTwo1 → another return address pushed.
- 3 AddTwo1 prints values of both variables, executes RET → returns to AddTwo.
- 4 AddTwo executes RET → returns to main.
- 5 main regains control, finishes execution, and exits.

Thank you ☺